Artificial Intelligence for EDA Tool Automation: A Review of AI-Driven VLSI Design Workloads

Pranav Jois
Roll No. 231EC237
Mobile No. 9686326473
pranavjois.231ec237@nitk.edu.in
Electronics & Communication Engineering Department
National Institute of Technology Karnataka, Surathkal

Rohit L
Roll No. 231EC248
Mobile No. 9482962815
rohitlalu.231ec248@nitk.edu.in
Electronics & Communication Engineering Department
National Institute of Technology Karnataka, Surathkal

Prateek Goel
Roll No. 231EC142
Mobile No. 9620204015
prateekgoel.231ec142@nitk.edu.in
Electronics & Communication Engineering Department
National Institute of Technology Karnataka, Surathkal

K Chandrasekaran
Professor
kc@nitk.edu.in
Computer Science & Engineering Department
National Institute of Technology Karnataka, Surathkal

***Abstract*—The growing complexity of modern VLSI systems has made Electronic Design Automation (EDA) tools essential for hardware development. However, existing workflows still require multiple tools, repetitive scripting, and extensive design-space exploration. Recent advances in artificial intelligence (AI), machine learning (ML), and large language models (LLMs) offer ways to automate these tasks. This paper reviews and compares fourteen studies on AI-driven EDA, covering ML-based prediction and optimisation, graph neural networks (GNNs), RTL generation, domain-adapted LLMs, and autonomous EDA agents. The studies are classified by AI technique, design stage, and a five-level automation scale introduced in this paper. Software engineering principles, including modularity, interoperability, testing, and reproducibility, guide the analysis of research gaps and the identification of future directions. The paper also presents a methodology for measuring how evaluation choices affect RTL generation pass@1, validated using synthetic models. The review highlights fragmented research, inconsistent evaluation practices, and the continued need for reliable verification of AI-generated designs.**

***Keywords*—Artificial Intelligence, VLSI, Machine Learning, Large Language Models, Electronic Design Automation, RTL Generation, Software Engineering**

I. Introduction

The continuous growth in the complexity of integrated circuits has made modern VLSI design increasingly dependent on Electronic Design Automation (EDA) tools. Contemporary digital systems integrate processors, memories, communication interfaces, accelerators, and specialised hardware blocks containing millions or billions of transistors. Designing and validating such systems manually is impractical. EDA tools, therefore, automate a large portion of the design process, including hardware description, logic synthesis, placement, routing, timing analysis, power analysis, verification, and physical implementation.

Despite the capabilities of modern EDA tools, the complete design process remains complex and iterative. A typical digital design requires several specialised tools, each with its own input formats, scripting interfaces, constraints, libraries, and report formats. Engineers must configure individual tools, execute scripts, analyse reports, modify design parameters, and repeat the process. Design space exploration further increases effort because many combinations of architectural, synthesis, and physical design parameters may need to be evaluated before acceptable power, performance, and area (PPA) are achieved.

Artificial intelligence has increasingly been investigated as a means of reducing this burden. Machine learning (ML) techniques have been applied to prediction, optimisation, design space exploration, and other EDA problems. Huang et al. survey ML applications across the EDA hierarchy and show how learning based methods have been applied at multiple stages of chip design [1]. Chen et al. examine ML methodologies for advanced IC design and identify the large design space and growing complexity of modern ICs as the central motivations for AI based methods [2]. Concrete results include deep reinforcement learning for chip floorplanning, which produced floorplans comparable or superior to human designs in power, performance, and area within hours [3], and ML models that estimate PPA directly from RTL without running synthesis [4].

Graph neural networks (GNNs) have attracted attention because circuits naturally possess graph structure. Gates, cells, nets, and their connectivity can be represented as

graphs, allowing graph based models to learn structural information. Sánchez et al. review the relationship between GNNs and EDA and classify existing work according to circuit representations, tasks, and model types [5].

More recently, large language models (LLMs) have introduced a different form of AI assisted hardware design. Rather than limiting AI to numerical prediction or optimisation, LLMs can process natural language specifications and generate hardware description language (HDL) code, scripts, documentation, and other design artefacts. Fine tuned open models for Verilog generation [6], [7], benchmarks for assessing them [8], and conversational co-design studies [9] have established RTL generation as an active research area. LLM use has also expanded from isolated code generation towards interaction with complete EDA workflows. ChatEDA investigates an LLM-powered autonomous agent that performs task planning, script generation, and execution with EDA tools as executors [10], while ChipNeMo studies domain adaptation of LLMs for industrial chip design [11]. Recent surveys broaden the analysis of LLMs across EDA, including code generation, verification, knowledge representation, and optimisation [12], [13].

These developments indicate a progression from conventional EDA automation towards AI assisted and increasingly AI-integrated workflows. However, the research is distributed across different EDA stages and AI techniques, and individual studies rarely use common benchmarks or metrics. A structured comparison is therefore useful for understanding which problems have been addressed, which advantages have been demonstrated, and which limitations remain.

This paper presents a survey and comparative analysis of AI-driven EDA automation research. Its objectives and contributions are as follows:

- To review the evolution of AI applications in EDA, from ML-based prediction to LLM-based agents.
- To classify existing approaches according to the AI technique, the VLSI design stage, and a five-level scale of automation.
- To compare the strengths and limitations of the approaches in a single consolidated table.
- To identify common research gaps across the existing studies.
- To examine how software engineering principles can improve the reliability and maintainability of AI-driven EDA workflows.
- To specify a methodology for measuring how much evaluation choices move the reported pass@1 of LLM-generated RTL, and to validate it on synthetic models with known effects.

A. Conventional VLSI Design Workflow

A conventional digital VLSI design process consists of multiple stages as seen in Fig. 1. The process generally begins with a system specification, which is converted into an RTL description using languages such as Verilog or SystemVerilog. Functional simulation and verification are then performed to determine whether the RTL satisfies the required behaviour. The RTL stage is particularly consequential because designers define precise behaviour there and make fine-grained decisions that ultimately determine PPA, yet the quality of an RTL design is conventionally assessed only by running the subsequent synthesis steps [4].

After functional verification, logic synthesis converts the RTL into a gate-level representation using a target standard cell library. Physical design subsequently includes floorplanning, placement, clock tree synthesis, and routing. Timing analysis, power analysis, design rule checking (DRC), and layout versus schematic (LVS) verification are performed before final manufacturing data is generated.

The stages are not strictly linear. A timing violation discovered after synthesis or physical implementation may require modifications to the RTL, synthesis constraints, or physical design parameters. Similarly, excessive area or power may trigger another iteration of the flow. The iterative nature of the flow makes automation particularly valuable. However, conventional automation is predominantly based on predefined scripts and tool specific commands. This provides reproducibility but requires significant domain knowledge.

B. Limitations of Traditional Automation

Tool heterogeneity. Complete flows may involve simulators, synthesis tools, place and route tools, timing analysis tools, and physical verification tools. These tools often use different command languages, constraint formats, and report formats, so engineers carry the burden of integration.

Expensive design space exploration. Numerous parameters affect PPA, timing closure, routability, and other objectives. Exhaustive exploration requires many computationally expensive EDA runs, and each run may take hours for large designs. Logic synthesis alone is a combinatorial optimisation problem in which the choice of transformation sequence influences the final netlist quality [14].

Static procedures. Traditional scripts follow predefined procedures. They do not naturally interpret unstructured design requirements or dynamically change the workflow based on information contained in generated reports.

No reuse of experience. Knowledge from previous design iterations is not automatically reused. Engineers rely on experience to select parameters or recognise patterns in reports, whereas scripts execute predetermined instructions.

These limitations have motivated research into ML based prediction and optimisation and, more recently, LLM based assistance and automation [1], [2], [12].

C. Role of Artificial Intelligence in EDA

AI applications in EDA can broadly be grouped into three categories:

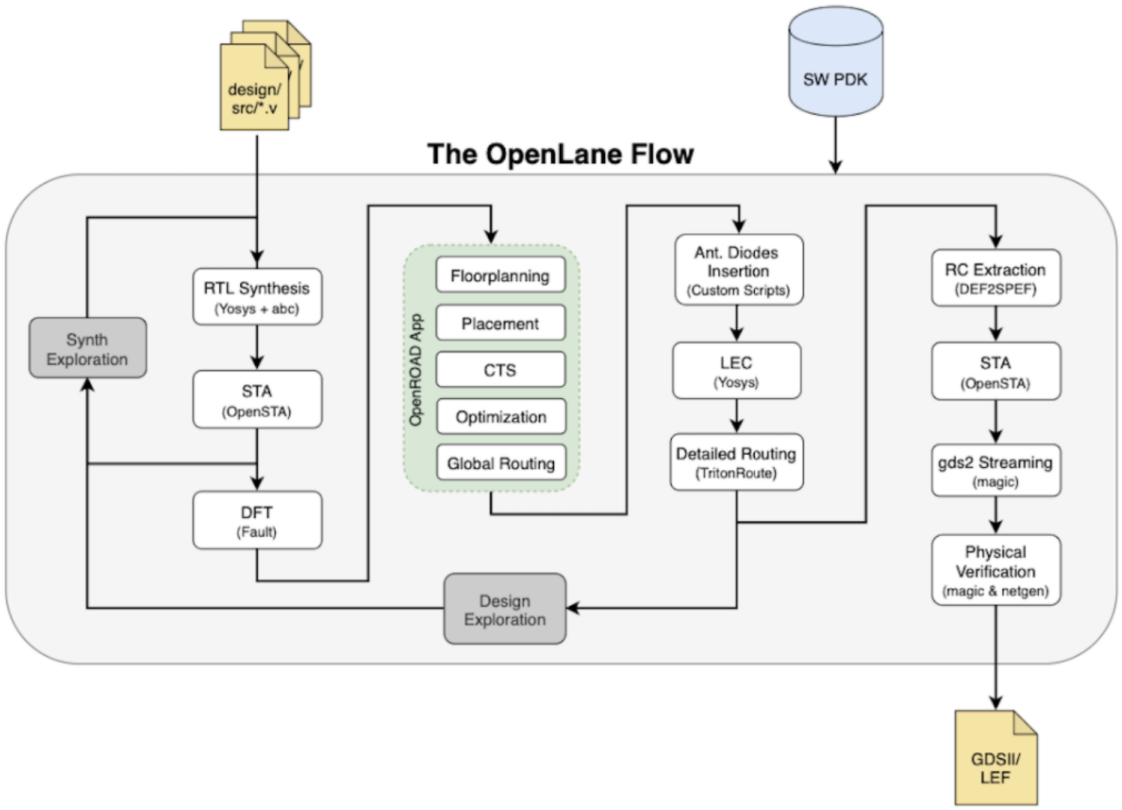


Fig. 1. Conventional digital VLSI design flow.

- **Prediction**: An AI model estimates a design property that would otherwise require a computationally expensive EDA operation, such as timing, power, area, routability, or other quality metrics.

- **Optimisation**: AI is used to search a large design space and identify promising configurations. ML and reinforcement learning (RL) techniques can guide such searches [1], [2]. In the RL formulation, a policy interacts with a design environment through states, actions, and rewards derived from quality metrics, and is trained to maximise the expected return.

- **Generation and interaction**: AI generates RTL, scripts, testbenches, constraints, explanations, or tool commands. LLM based approaches primarily belong to this category, although they can also support optimisation and analysis.

These categories are complementary rather than mutually exclusive. A future EDA workflow may use different AI techniques for different tasks rather than relying on a single model for the entire design process.

D. Software Engineering Perspective

Since AI driven EDA couples probabilistic models with deterministic tools, the resulting workflow is a software system in its own right. This paper, therefore, evaluates the literature against six software engineering principles:

Modularity is the decomposition of the workflow into components with well defined responsibilities.
Abstraction hides tool specific detail behind stable interfaces.
Interoperability is the ability of components to exchange data through standard formats.
Testing is the systematic checking of every generated artefact against an independent oracle.
Reproducibility is the ability to regenerate a result from recorded inputs, versions, and settings.
Human oversight is the explicit placement of review and approval points.

The remainder of the paper is organised as follows. Section II reviews the literature and presents the comparative analysis and identifies research gaps. Section III specifies and validates a methodology for measuring evaluation sensitivity. Section IV concludes the discussion.

II. Literature Review

The reviewed research is subdivided into five groups: ML for EDA, GNNs for EDA, LLM assisted RTL generation, domain adapted LLM assistants, LLM surveys, and autonomous EDA agents. Although the works look similar at first glance, differences in their implementation strategies, evidence, and degree of automation warrant grouping them

separately.

Each work is classified along three dimensions: the **AI technique** (ML, GNN, RL, LLM, LLM agent), the **design stage** (RTL, logic synthesis, physical design, or the whole flow), and the **degree of automation**. The last dimension reflects who performs the execution and who verifies the result. Five levels are:

- **L0 (Scripted)**: Fixed scripts execute the flow, the engineer interprets reports and decides.
- **L1 (Advisory)**: A model predicts or ranks, the engineer decides and acts.
- **L2 (Generative)**: A model produces an artefact (RTL, script, placement) and the engineer reviews and verifies it.
- **L3 (Supervised agent)**: A model plans and invokes tools across several steps, the engineer supervises and approves.
- **L4 (Autonomous)**: Closed loop of planning, execution, and verification without a human in the loop.

A. Machine Learning for EDA

ML based EDA research predates the recent growth of LLMs. Huang et al. provide a broad survey of ML techniques applied throughout the EDA hierarchy [1]. Their review shows that ML can learn relationships between design characteristics and outcomes, which makes it useful both for predicting results and for guiding optimisation. A major advantage is the ability to approximate expensive evaluations. If a model accurately predicts a design metric, designers can reduce the number of full EDA executions required during exploration, thereby allowing ML to serve as a surrogate for selected operations.

Chen et al. analyse ML methods from both the IC design problem perspective and the ML method perspective [2]. Their analysis emphasises the large and complex design space of advanced IC design. Different learning methods suit different problems, and the usefulness of an ML model depends strongly on the quality and relevance of its training data.

Prediction at RTL: MasterRTL is a representative primary study of early prediction [4]. Addresses the problem that assessing the quality of an RTL design typically requires time consuming synthesis, which impedes optimization at the early RTL stage. The framework converts HDL code into a bit-level representation, the simple operator graph (SOG), built only from single-bit simple operators, so that different design types and styles are unified and the representation resembles the target gate-level netlist. Separate ML models then estimate timing, power, and area, and the timing model explicitly captures the critical path and the delay between register pairs. The authors also extend the framework to post-placement PPA. MasterRTL illustrates both the promise and the dependence of surrogate models on data: their value depends on how well the training designs cover those encountered later.

Data for ML-guided synthesis: OpenABC-D addresses the absence of standard datasets for ML guided logic synthesis [14]. It was produced by synthesising open-source hardware designs with random synthesis recipes using an open-source synthesis tool, yielding And-Inverter Graphs (AIGs) with labels such as node count, longest path, area, and delay, and defining learning problems such as predicting the quality of a recipe's result. Datasets of this kind are a prerequisite for comparable evaluation. Still, a dataset generated using a single open-source tool and a limited number of designs does not, by itself, represent industrial flows.

Optimisation in Physical Design: Mirhoseini et al. pose chip floorplanning as an RL problem and use an edge-based graph convolutional network to learn transferable representations of the chip [3]. In under six hours, the method generates floorplans reported to be superior to or comparable to those of human designers in power, performance, and area, and it was used to design the next generation of Google's AI accelerators. This work is the strongest example in the surveyed set of AI optimisation that has reached production use. Its scope, however, is a single stage, floorplanning, and the approach inherits the computational cost and data requirements of RL training.

Taken together, the literature suggests that ML is particularly valuable for specialised EDA tasks. Still, the resulting models are designed for specific problems rather than for end-to-end automation, and each depends on labelled data produced by the very tools it is meant to relieve.

B. Graph Neural Networks in EDA

Circuit designs have an inherent graph structure. Components can be represented as nodes, and electrical or logical connections as edges, making GNNs particularly relevant to circuit analysis.

Sánchez et al. provide a comprehensive survey of GNN applications in EDA and map existing approaches to the EDA pipeline [5]. The review considers graph representations of circuits, RTL structures, and netlists, as well as various prediction and optimisation tasks. Graph representations also underpin several primary studies discussed above: OpenABC-D provides AIGs in a format directly usable by graph based ML frameworks [14], the SOG of MasterRTL is a graph structured design representation [4], and the RL floorplanner relies on an edge based graph network [3].

The main advantage of GNNs is their ability to incorporate structural information. Conventional feature based ML methods require carefully engineered features, whereas graph based models can directly process relationships between circuit elements. However, GNN based EDA approaches also face challenges. The representation of a circuit varies across

design stages (RTL graph, AIG, netlist, placement graph), and the performance of a trained model may depend on the circuit type, technology, and target task. Generalisation across substantially different designs, therefore, remains an important consideration.

C. LLM Assisted RTL Generation

The emergence of LLMs has introduced natural-language interaction into hardware design. The most active strand is RTL generation, in which a model produces Verilog from a specification or completes partial code.

Fine-tuned models: VeriGen fine-tunes pre-trained LLMs on Verilog code collected from GitHub and textbooks, using models ranging from 345M to 16B parameters [6]. Its fine-tuned CodeGen-16B model performs 1.1% better than GPT-3.5-turbo overall and improves syntactically correct Verilog generation by 41% over the original model. RTLCoder focuses on a 7B-parameter model trained specifically for RTL generation and provides its dataset and training method publicly [7]. It is reported to outperform GPT-3.5 on all tested RTL benchmarks and GPT-4 on the VerilogEval Machine benchmark. These studies show that training models on hardware-specific data can improve RTL generation while offering greater flexibility and control.

Benchmarking: VerilogEval proposes a benchmark of 156 problems drawn from the HDLBits instructional website, ranging from simple combinational circuits to complex finite-state machines [8]. Completions are tested for functional correctness by comparing simulation outputs with a golden solution, and the authors show that supervised fine-tuning on synthetic problem–code pairs generated by an LLM can improve generation capability. Functional correctness in such benchmarks is commonly summarised by the *pass@k* metric.

Conversational design: Chip-Chat examines how conversational LLMs behave in hardware design [9]. Using a suite of eight benchmarks, the study found that models performed best when used interactively. In a longer case study, a hardware engineer co-architected an 8-bit accumulator-based microprocessor with an LLM, and the design was sent to tape-out on a SkyWater 130 nm shuttle, which the authors believe is the first wholly AI-written HDL sent for tape-out.

LLM based RTL generation can reduce repetitive coding effort and provide a more accessible interface between high-level specifications and HDL implementations. However, code generation should not be equated with functional correctness. A generated RTL module may be syntactically valid while implementing incorrect behaviour, and benchmark success on small, self-contained modules does not establish that a model can handle large hierarchical designs or meet PPA targets. Therefore, simulation, formal verification, synthesis checks, and other validation methods remain necessary. This leads to an important observation from the literature: LLMs are useful as design assistants, but deterministic verification tools remain necessary to establish hardware correctness.

D. Domain-Adapted Assistants and LLM Surveys

ChipNeMo explores LLMs for industrial chip design through domain adaptation: a custom tokeniser, domain adaptive continued pre-training, supervised fine tuning with domain specific instructions, and domain-adapted retrieval models [11]. It is evaluated on three applications: an engineering-assistant chatbot, EDA script generation, and bug summarisation and analysis. The adaptation techniques yield significant improvements over general purpose base models and enable a model size reduction of up to 5× with similar or better performance across a range of design tasks. However, the authors acknowledge a remaining gap to ideal results. A further motivation is that customising LLMs avoids the need to send proprietary design data to third-party APIs. ChipNeMo is notable because script generation and report analysis move LLM use beyond RTL code towards the workflow around EDA tools.

Pan et al. survey LLM research in EDA and examine applications across several stages of electronic design [12]. Their work shows that LLM research is expanding beyond RTL generation towards verification, design understanding, optimisation, and physical design related tasks. He et al. provide another recent survey that categorises LLM based EDA research into code generation, verification and debugging, knowledge representation and retrieval, and optimisation [13]. The breadth of this classification indicates that LLMs are increasingly regarded as general-purpose interfaces for hardware design information rather than merely code generators. Both surveys highlight persistent issues with reliability, domain specific datasets, evaluation, hallucination, and the difficulty of translating general language model capabilities into trustworthy hardware design decisions [12], [13].

E. Autonomous EDA Agents

ChatEDA extends LLM based assistance beyond RTL generation [10]. It is an autonomous EDA agent powered by an LLM, AutoMage, in which EDA tools serve as executors. The agent performs task planning, script generation, and task execution, and targets the flow from RTL to GDSII. The authors report that AutoMage outperforms GPT-4 and similar LLMs on their evaluation.

From a software engineering perspective, this represents an important transition from manually coordinating multiple software tools to an intelligent controller that interacts with them via a higher level workflow, corresponding to L3 on the scale. The planner, the script generator, and the executor form distinct modules, and EDA tools are accessed through an interface rather than operated by hand.

However, greater autonomy also introduces additional failure modes. An incorrectly generated command, an incorrect interpretation of a tool report, or an inappropriate task sequence can propagate an error to subsequent stages. A script that executes without error may still apply the wrong constraint, so the absence of a tool error is not evidence of correctness. Increasing the degree of automation, therefore, increases the importance of validation, error handling, logging, and controlled interfaces. No surveyed work demonstrates L4 operation, in which the verification of each step is also automated.

F. Comparative Analysis

Table I consolidates the strengths and drawbacks of all reviewed works. The comparison highlights four key findings. First, existing research focuses mainly on RTL generation and logic synthesis, while physical design, verification, and sign-off receive less attention. Second, studies with lower levels of automation generally provide clearer benchmark-based results, whereas more autonomous approaches have limited evidence of performance against conventional engineering workflows. Third, most AI generated designs still depend on external verification through testbenches, reference models, or human review. Finally, many studies face similar limitations, including dependence on domain specific data, limited generalisation, and a lack of coordination across different design stages. These findings highlight the need for more integrated, reliable, and thoroughly evaluated AI driven EDA workflows.

G. Research Gaps

The literature reveals that AI-based EDA research is advancing rapidly, but the research remains distributed across different tasks. ML approaches are effective for specialised prediction and optimisation problems [1]–[4]. GNN approaches exploit circuit structure [5]. LLMs demonstrate potential for RTL generation [6], [7], workflow interaction [10], and broader EDA assistance [11]–[13]. However, several gaps become apparent when these studies are considered together.

Cross stage integration: A model designed for RTL generation does not automatically understand physical design constraints, and a stand-alone physical design optimiser does not understand high level functional requirements. Each surveyed model operates on a single representation (HDL text, an AIG, or a placement graph), and no surveyed work learns a shared representation that propagates information from specification to layout. Because the stages interact through iteration, local optimisation does not guarantee global improvement.

Non uniform evaluation: Different studies use different datasets, benchmarks, technology libraries, EDA tools, and performance metrics. VerilogEval and the VeriGen problem set, for example, are distinct benchmarks [6], [8], so reported advantages over GPT 3.5 are not directly comparable.

Reproducibility: AI-based EDA results may depend on model versions, prompts, training data, random seeds, sampling temperature, tool versions, and implementation settings. Closed commercial models can change without notice, and industrial studies based on proprietary data cannot be independently repeated [11]. Open datasets and models [7], [14] mitigate this concern but are not yet the norm.

Verification and validation: The probabilistic nature of generative AI makes independent validation particularly important. Generated RTL may be syntactically valid but functionally wrong, and a generated script may run to completion while applying an unintended constraint. Few studies integrate formal equivalence checking or coverage-driven verification into the generation loop.

III. Methodology

This section specifies a one factor at a time methodology for LLM generated RTL and validates the measurement pipeline on synthetic models whose behaviour is known exactly.

A. Problem Subset

The experiment uses the spec-to-RTL dataset from VerilogEval [8], which contains 156 problems. A fixed repository commit is used to maintain dataset consistency. Problems are excluded if their reference solutions fail the benchmark testbench, are not accepted by the Yosys front end, produce no cells during synthesis, or do not contain the standard opening sentence required for the prompt comparison. These filtering steps remove 28 problems, leaving 128 eligible problems. From these, 60 problems are selected using a fixed random seed. The selected problem list is stored in the repository, and a unit test checks that the same list can be reproduced.

B. Factors and Configurations

The experiment examines the effects of changes in random seeds, prompt wording, code extraction rules, module naming, and synthesis settings. Changes to the seed and prompt require new model responses, while extraction and naming changes are applied to the same saved responses to isolate their effects. The baseline configuration uses the original prompt, standard code extraction, and strict module naming, with ten samples per problem at a fixed set of generation settings. Each additional configuration changes one factor at a time to make the results easier to compare. A separate pair of runs at zero temperature is used to check whether the model produces consistent responses. Different synthesis scripts are also tested to examine their effect on the reported cell counts. Across both models, the complete experimental design requires 2,640 generations.

C. Pipeline and Tools

The pipeline consists of five stages: code generation, code extraction, compilation and simulation, synthesis, and result analysis as shown in Fig. 2. Code is generated via an OpenAI-compatible endpoint and tested with Icarus Verilog, with a 30

TABLE I
COMPARISON OF THE REVIEWED LITERATURE: STRENGTHS AND DRAWBACKS

Ref.	Technique and stage	Main contribution	Strengths	Drawbacks	Level
[1]	ML survey; whole EDA hierarchy	Maps ML applications across design stages	Broad coverage; shows ML as surrogate for expensive evaluations	Predates LLM agents; no common benchmark across surveyed methods	–
[2]	ML survey; advanced IC design	Analyses methods from problem and method perspectives	Links problem characteristics to suitable learning methods	Outcome depends on training-data quality; models are task-specific	–
[5]	GNN survey; EDA pipeline	Classifies circuit representations, tasks, and model types	Exploits circuit structure without hand-crafted features	Representations vary by stage; generalisation across designs and technologies	–
[4]	ML on a bit-level graph (SOG); RTL	Pre-synthesis timing, power, and area estimation	Avoids synthesis for early feedback; unified representation across design styles	Needs labelled data from full flows; accuracy tied to training-design coverage	L1
[14]	Dataset of AIGs; logic synthesis	870 000 AIGs from 1500 recipes on 29 designs	Standard data for ML-guided synthesis; graph-ready format	One open-source tool and limited designs; not representative of industrial flows	–
[3]	RL with graph network; floorplanning	Fast automated floorplans	Results comparable or superior to human designs in power, performance, and area; transfers across chips	Single stage; heavy training compute; cross-stage effects not modelled	L2
[6]	Fine-tuned LLM; RTL	Verilog completion with open fine-tuned models	Open models competitive with GPT-3.5-turbo; large gain in syntactic correctness	Functional correctness limited; custom problem set; no PPA assessment	L2
[8]	Benchmark; RTL	156 problems with automatic functional checking	Reproducible simulation-based scoring against golden solutions	Small, self-contained modules; no synthesisability or PPA criteria	–
[7]	Open LLM and dataset; RTL	7B model with fully open dataset and algorithm	Strong benchmark results; open and lightweight; privacy-friendly	Benchmark-level modules; correctness still needs independent verification	L2
[9]	Conversational LLM; RTL to tape-out	Human–LLM co-design of an 8-bit microprocessor	Practical evidence from a fabricated design; effective when interactive	Single small design; depends on engineer skill and manual checking	L2
[11]	Domain-adapted LLM; assistant, scripts, bug triage	Domain adaptation for industrial chip design	Up to 5× smaller model at similar or better performance; keeps data in-house	Gap to ideal results remains; proprietary data limits reproduction	L1–L2
[10]	LLM agent; RTL to GDSII	Task planning, script generation, and tool execution	Automates multi-tool flows from natural-language intent	Errors can propagate across stages; needs strong validation and logging	L3
[12], [13]	LLM surveys; multiple stages	Broad classification of LLM use in EDA	Cover generation, verification, retrieval, and optimisation	Fast-moving field; reliability, datasets, and evaluation unresolved	–

second simulation time limit. A sample is considered correct only if the testbench reports no mismatches and no errors or timeouts occur. The pipeline also records different failure types, such as missing code, compilation errors, and simulation failures. Synthesis is performed with Yosys, and the reported cell count serves as a basic measure of design complexity. Since no cell library is used, this count does not represent actual area, timing, or power. A run manifest records the tool versions, model details, hardware, prompts, sampling settings, and experimental configurations to support reproducibility.

D. Metrics and Decision Rule

The evaluation uses several metrics to study how changes in prompts, random seeds, and code evaluation settings affect RTL generation. The pass rate measures the percentage of problems for which the generated code passes the required checks. Results are collected across multiple runs to understand how much the pass rate varies between runs. This variation is important because results from a single run may not represent the model's typical performance.

The observed variation as seen in Fig 3 is also compared with the variation expected from the individual problem results. If the difference is too large, the experiment is examined before the results are reported. Confidence intervals are used to estimate the uncertainty in the measurements and to compare different configurations.

Each factor is compared against a baseline configuration to determine its effect on the pass rate. An effect is considered meaningful when it is sufficiently large relative to the variation between runs. Smaller changes that can be distinguished from zero are reported separately, while effects that cannot be reliably distinguished from normal variation are treated as inconclusive.

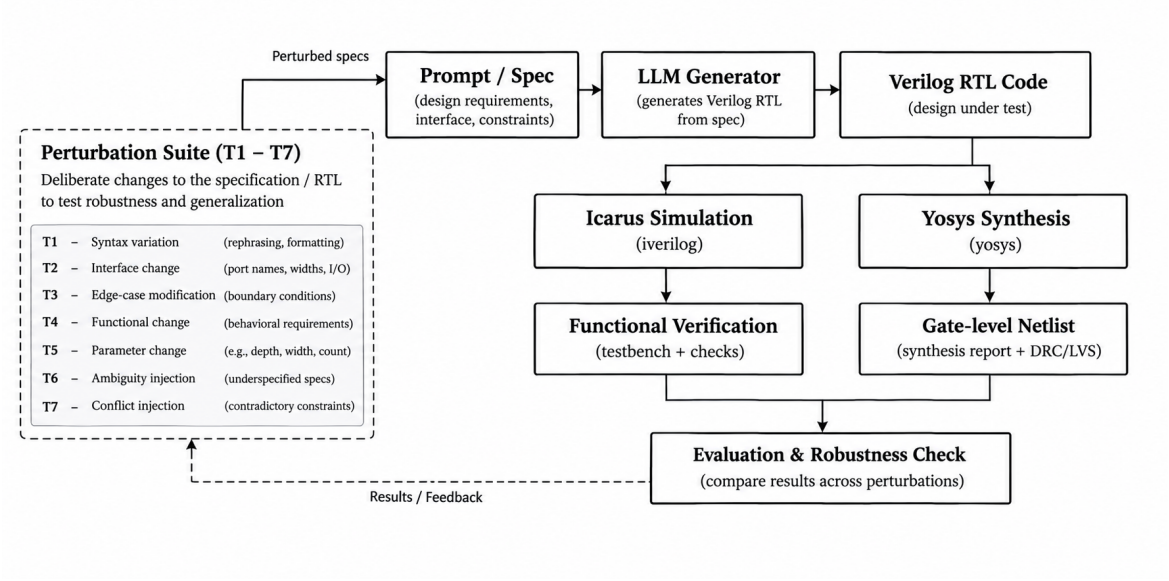


Fig. 2. Block Diagram representation of AI driven RTL to GDSII robustness workflow

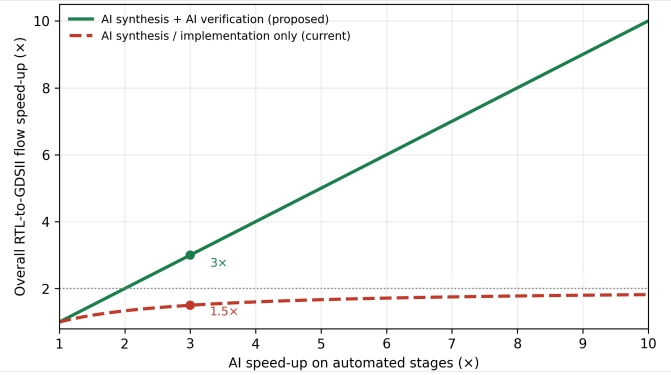


Fig. 3. AI EDA Verification Speedup Comparison

The analysis also examines changes in individual problem outcomes, since an overall pass rate can remain similar even when different problems pass or fail. For evaluation settings, the analysis records both improvements and regressions. For prompt changes, it compares observed outcome changes with those attributable to random variation between runs. When comparing two models, the analysis considers both the overall performance difference and how consistently one model outperforms the other. Finally, determinism is assessed by checking whether repeated responses are identical and whether they produce the same pass or fail outcomes.

E. Validation on Synthetic Models

Design: Since the effects of different factors on real LLMs are not yet known, the pipeline was first tested using two synthetic models with predefined behaviour. These models either return the correct reference solution or an empty module with the correct ports, based on predefined success probabilities. The two models were designed to have different success rates and responses to changes in prompts. The experiment also introduced common formatting issues, such as placing the testbench before or after the code, omitting code fences, and using a different module name. These cases were used to check whether the pipeline could correctly extract and evaluate the generated code. One model also occasionally added random comments at zero temperature to test the consistency of responses.

Pipeline checks: Of the 22 acceptance checks, 20 passed, while two checks requiring a running LLM server were skipped. An oracle test that always returned the correct solution passed all 720 evaluation cases. Three negative tests that used incomplete modules or plain text without code correctly identified all cases as failures. Running the same evaluation with one and two worker processes produced identical results. A statistical simulation also showed that the pipeline's measurements were close to the expected values.

Determinism and gate counts: At zero temperature, model A produced identical responses across all 60 tests, while model B produced identical responses in 48 cases. However, both models produced the same pass-or-fail outcome for every test. The gate-count stage was also tested, but it provided no evidence about the sensitivity of real designs because every successful synthetic sample used the reference solution. A preliminary test on a population counter showed that different synthesis scripts could produce substantially different cell counts, highlighting the need for consistent evaluation settings.

F. Status of the Real Model Experiment

The experimental factors, evaluation metrics, and decision rules were defined before testing. The planned real-world model experiment uses two open-source models with around 7–8 billion parameters, running locally via an OpenAI-compatible endpoint. However, this experiment has not yet been completed, and the two remaining server checks must pass before it can begin. Therefore, the current results validate the pipeline using only synthetic models. They do not establish how sensitive real LLMs are to changes in prompts, random seeds, or code extraction settings.

IV. Conclusion

Artificial Intelligence is creating new opportunities to automate and improve Electronic Design Automation (EDA) workflows. This paper reviewed studies covering machine learning, graph neural networks, large language models, and autonomous EDA agents. The proposed five-level automation scale helped compare their capabilities, from scripted workflows to autonomous design flows. The review shows that, although AI has made progress in tasks such as RTL generation, design optimisation, and floorplanning, most existing approaches remain focused on individual stages of the design process. Differences in evaluation methods and the limited use of synthesis and physical design metrics also make direct comparisons difficult.

A key finding is that generating functionally correct RTL is not sufficient for reliable hardware design. AI generated code must also pass synthesis and verification checks before it can be considered useful in practical workflows. The `rtlsens` framework explores this issue by combining functional simulation with logic synthesis and examining how prompt changes affect RTL generation. Its evaluation highlights the importance of checking synthesizability and studying the sensitivity of generated designs to evaluation conditions.

Future research should focus on the following areas:

- Integrating simulation, synthesis, and verification into closed loop AI workflows to detect and correct errors.
- Developing standardised benchmarks that evaluate functional correctness, area, power, and timing across realistic designs.
- Improving interoperability, reproducibility, and security through modular architectures, standardised interfaces, controlled execution environments, and detailed logging.
- Studying human-AI collaboration and developing privacy preserving models suitable for industrial use.

Overall, combining AI techniques with software engineering practices is essential for building reliable, maintainable, and scalable AI driven EDA systems. Progress in these areas can help move AI-EDA from isolated experimental tools toward practical assistance across the VLSI design flow, while retaining verification and human oversight for critical design decisions.

References

[1] G. Huang, J. Hu, Y. He, J. Liu, M. Ma, Z. Shen, J. Wu, Y. Xu, H. Zhang, K. Zhong, X. Ning, Y. Ma, H. Yang, B. Yu, H. Yang, and Y. Wang, "Machine learning for electronic design automation: A survey," *ACM Transactions on Design Automation of Electronic Systems*, vol. 26, no. 5, pp. 1–46, Jun. 2021.

[2] T. Chen, G. L. Zhang, B. Yu, B. Li, and U. Schlichtmann, "Machine learning in advanced IC design: A methodological survey," *IEEE Design & Test*, vol. 40, no. 1, pp. 17–33, Feb. 2023.

[3] A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. Songhori, S. Wang, Y.-J. Lee, E. Johnson, O. Pathak, A. Nova *et al.*, "A graph placement methodology for fast chip design," *Nature*, vol. 594, no. 7862, pp. 207–212, Jun. 2021.

[4] W. Fang, Y. Lu, S. Liu, Q. Zhang, C. Xu, L. W. Wills, H. Zhang, and Z. Xie, "MasterRTL: A pre-synthesis PPA estimation framework for any RTL design," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2023.

[5] D. Sánchez, L. Servadei, G. N. Kiprit, R. Wille, and W. Ecker, "A comprehensive survey on electronic design automation and graph neural networks: Theory and applications," *ACM Transactions on Design Automation of Electronic Systems*, vol. 28, no. 2, pp. 1–27, Feb. 2023.

[6] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "VeriGen: A large language model for Verilog code generation," *ACM Transactions on Design Automation of Electronic Systems*, 2024.

[7] S. Liu, W. Fang, Y. Lu, J. Wang, Q. Zhang, H. Zhang, and Z. Xie, "RTLCoder: Fully open-source and efficient LLM-assisted RTL code generation technique," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 4, pp. 1448–1461, Apr. 2025.

[8] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "VerilogEval: Evaluating large language models for Verilog code generation," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2023.

[9] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-chat: Challenges and opportunities in conversational hardware design," in *Proc. ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*, 2023.

[10] H. Wu, Z. He, X. Zhang, X. Yao, S. Zheng, H. Zheng, and B. Yu, "ChatEDA: A large language model powered autonomous agent for EDA," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 10, pp. 3184–3197, Oct. 2024.

[11] M. Liu *et al.*, "ChipNeMo: Domain-adapted LLMs for chip design," arXiv:2311.00176, 2023.

[12] J. Pan, G. Zhou, C.-C. Chang, I. Jacobson, J. Hu, and Y. Chen, "A survey of research in large language models for electronic design automation," *ACM Transactions on Design Automation of Electronic Systems*, vol. 30, no. 3, pp. 1–21, Feb. 2025.

[13] Z. He, Y. Pu, H. Wu, T. Qiu, and B. Yu, "Large language models for EDA: Future or mirage?" *ACM Transactions on Design Automation of Electronic Systems*, vol. 30, no. 6, pp. 1–53, Oct. 2025.

[14] A. B. Chowdhury, B. Tan, R. Karri, and S. Garg, "OpenABC-D: A large-scale dataset for machine learning guided integrated circuit synthesis," arXiv:2110.11292, 2021.